Foundations of Artificial Intelligence

Part V: Natural Language Processing (Introductory)

Department of Computer Science

Lecture Roadmap

1. Text Preprocessing
2. Bag-of-Words Representation
3. Basic Sentiment Analysis
4. NLP Applications

Learning Objectives

By the end of this lecture, you should be able to:

- Apply basic text preprocessing techniques: tokenization, lowercasing, stopword removal, stemming.
- Build and interpret Bag-of-Words representations for text.
- Implement basic sentiment analysis using word-level patterns.
- Discuss real-world NLP applications: chatbots, translation, and speech systems.
- Recognize the role of language models and the limitations of simple approaches.

Text Preprocessing

Why Preprocessing Matters

Raw text is messy:

- Mixed case, punctuation, special characters.
- Irrelevant words (“the”, “a”, “is”).
- Different forms of the same word (“run”, “runs”, “running”).

Preprocessing standardizes text so that algorithms see meaningful units consistently.

Core steps:

1. Tokenization
2. Lowercasing and normalization
3. Stopword removal
4. Stemming/lemmatization

Tokenization

Goal: Split text into individual words (tokens).

Simple approach: split on whitespace and punctuation.

- Input: ``Hello, world! How are you?''
- Output: [``Hello'', ``world'', ``How'', ``are'', ``you'']

Challenges:

- Contractions: ``don't'' $\rightarrow$ keep as one or split?
- Hyphenated words: ``state-of-the-art''.
- URLs, emails, numbers: preserve or normalize?

Most tools use regex-based or language-aware tokenizers.

Lowercasing and Normalization

Lowercasing: Convert all text to lowercase.

- Reduces vocabulary size.
- Treats ``Hello'', ``hello'', ``HELLO'' identically.

Other normalizations:

- Remove accents: ``café'' $\rightarrow$ ``cafe''.
- Replace numbers with placeholders: ``123 Main St'' $\rightarrow$ ``[NUM] Main St''.
- Expand contractions: ``don't'' $\rightarrow$ ``do not''.

Trade-off: lose information (e.g., proper nouns) for uniformity.

Stopword Removal

Stopwords: Common words with little discriminative power.

- Articles: “a”, “an”, “the”
- Pronouns: “I”, “you”, “he”, “she”
- Prepositions: “in”, “on”, “at”, “to”
- Conjunctions: “and”, “or”, “but”

Example:

- Before: “The cat is on the mat and it is very soft.”
- After: “cat mat soft”

Benefit: Reduces noise and speeds up processing.

Caveat: Domain-dependent; task may require keeping some stopwords.

Stemming and Lemmatization

Stemming: Remove suffixes to get word root.

- Input: “running”, “runs”, “ran”, “runner”
- Output (Porter Stemmer): “run”, “run”, “ran”, “runner”
- Heuristic-based; may produce non-words.

Lemmatization: Reduce to canonical dictionary form (lemma).

- Input: “running”, “runs”, “ran”, “runner”
- Output: “run”, “run”, “go”, “run”
- Requires linguistic knowledge; more accurate but slower.

Trade-off: Stemming is fast but crude; lemmatization is precise but needs external resources.

Preprocessing Pipeline Example

Step	Result
Original	“The analysis of running cats and dogs is fascinating!”
Tokenize	[“The”, “analysis”, “of”, “running”, “cats”, “and”, “dogs”, “is”, “fascinating”, “!”]
Lowercase	[“the”, “analysis”, “of”, “running”, “cats”, “and”, “dogs”, “is”, “fascinating”, “!”]
Remove punct.	[“the”, “analysis”, “of”, “running”, “cats”, “and”, “dogs”, “is”, “fascinating”]
Remove stopwords	[“analysis”, “running”, “cats”, “dogs”, “fascinating”]
Stem	[“analysi”, “run”, “cat”, “dog”, “fascin”]

Bag-of-Words Representation

From Tokens to Vectors

Bag-of-Words (BoW): Represent text as unordered collection of word counts.

Key idea: ignore order, focus on word frequency.

Example:

- Document 1: "I love machine learning"
- Document 2: "I love cats and dogs"
- Vocabulary: {I, love, machine, learning, cats, dogs, and}

BoW vectors (counts):

- Doc 1: $[1, 1, 1, 1, 0, 0, 0]$
- Doc 2: $[1, 1, 0, 0, 1, 1, 1]$

BoW Variants: TF and TF-IDF

Term Frequency (TF): Count of word in document.

$$\text{TF}(w, d) = \text{count}(w, d)$$

Problem: Common words dominate. Longer documents have higher counts.

TF-IDF: Weight by inverse document frequency.

$$\text{TF-IDF}(w, d) = \text{TF}(w, d) \times \log\left(\frac{N}{|\{d : w \in d\}|}\right)$$

where N is total documents and denominator counts how many documents contain w.

Intuition: Rare words in a document get high weight.

BoW Strengths and Limitations

Strengths:

- Simple to implement and understand.
- Works well with many classifiers (Naive Bayes, SVM, etc.).
- Computationally efficient.

Limitations:

- **Order-independent:** "dog bites man" and "man bites dog" are identical.
- **Loses context:** Single words may be ambiguous ("bank": financial or riverside?).
- **Sparse representations:** vocabulary can be very large (high-dimensional).
- **No semantic similarity:** "dog" and "cat" are unrelated vectors.

Word embeddings (e.g., Word2Vec, GloVe) address some limitations.

Document Similarity with BoW

Goal: Measure how similar two documents are.

Cosine Similarity:

$$\cos(\theta) = \frac{\vec{d}_1 \cdot \vec{d}_2}{|\vec{d}_1| \cdot |\vec{d}_2|}$$

where $\vec{d}_1, \vec{d}_2$ are BoW or TF-IDF vectors.

Range: $[-1, 1]$ (or $[0, 1]$ for non-negative vectors).

- $1 =$ identical direction (very similar).
- $0 =$ orthogonal (no similarity).
- $-1 =$ opposite (rare in BoW).

Application: Information retrieval, plagiarism detection, document clustering.

Basic Sentiment Analysis

What Is Sentiment Analysis?

Goal: Determine the emotional tone or opinion in text.

Common task: Binary classification.

- Positive vs. Negative
- Example: product reviews, social media posts.

More nuanced: Multi-class (positive, neutral, negative) or regression (score 1–5).

Why it matters:

- Brand monitoring.
- Customer feedback analysis.
- Social listening.

Lexicon-Based Approach

Idea: Use a sentiment lexicon (dictionary of words with known polarity).

Example lexicon:

Positive	good, great, excellent, love, amazing
Negative	bad, terrible, hate, awful, disappointing

Algorithm:

1. Tokenize and preprocess text.
2. Count positive and negative words.
3. Assign label: positive if more positive, else negative.

Example: "This movie is great and I loved it!"

- Positive words: great, loved (count $= 2$).
- Negative words: none (count $= 0$).
- Result: Positive.

Challenges in Sentiment Analysis

Negation: "I did **not** like this movie."

- "like" is positive, but negated.
- Simple lexicon approach fails.

Sarcasm: "Oh, great, another traffic jam."

- "great" is positive, but context is negative.
- Requires understanding intent.

Domain-specific language: "sick" can be negative or positive (slang).

Ambiguous words: "interesting" could be positive or negative depending on context.

Advanced methods: machine learning, neural networks, contextual embeddings.

Feature-Based Sentiment Analysis

Extend BoW with engineered features:

- **Word polarity:** sentiment score of each word (e.g., AFINN, TextBlob).
- **Negation handling:** flip polarity if preceded by negation.
- **Capitalization:** "AMAZING" might indicate strong emotion.
- **Punctuation:** multiple exclamation marks "!!!" suggest intensity.
- **Part-of-speech:** adjectives often carry opinion.

Workflow:

1. Extract features from preprocessed text.
2. Train classifier (Naive Bayes, SVM, Logistic Regression).
3. Predict sentiment on new reviews.

Sentiment Analysis Performance Metrics

Classification metrics:

- **Accuracy:** fraction of correct predictions.
- **Precision:** of predicted positives, how many are truly positive.
- **Recall:** of truly positive cases, how many are found.
- **F1-score:** harmonic mean of precision and recall.

Example confusion matrix (binary):

	Predicted Pos	**Predicted Neg**
Actual Pos	TP	FN
Actual Neg	FP	TN

$$\text{Precision} = \frac{\text{TP}}{\text{TP}+\text{FP}}, \quad \text{Recall} = \frac{\text{TP}}{\text{TP}+\text{FN}}$$

NLP Applications

Real-World NLP Applications

NLP powers many systems we use daily:

- **Chatbots & Virtual Assistants:** Siri, Alexa, ChatGPT.
- **Machine Translation:** Google Translate, DeepL.
- **Speech Recognition & Synthesis:** Dictation, text-to-speech.
- **Information Retrieval:** Search engines, document ranking.
- **Named Entity Recognition:** Extract people, places, organizations.
- **Question Answering:** Systems that find answers from text.
- **Text Summarization:** Automatic document or article condensing.

Modern systems often combine NLP with neural networks and large language models.

Chatbots

Goal: Simulate conversation with a human via text (or speech + NLP).

Architecture levels:

- **Rule-based:** Pattern matching, lookup tables. Limited but predictable.
- **ML-based:** Learns from training data. Intent classification + response generation.
- **Neural (Sequence-to-Seq):** Encoder-decoder models, transformer-based (e.g., GPT).

Challenge: Understanding context and maintaining conversation flow.

Current frontier: Large language models with few-shot learning enable more natural dialogue.

Machine Translation

Goal: Translate text from one language to another.

Historical approaches:

- **Rule-based:** Hand-coded grammar rules and bilingual dictionaries.
- **Statistical MT (SMT):** Learn probabilities from aligned parallel texts.
- **Neural MT (NMT):** Encoder-decoder with attention mechanisms.

Key challenge: Structural differences between languages, idioms, context.

Sequence-to-Sequence with Attention:

- Encoder processes source language.
- Decoder generates target language.
- Attention focuses on relevant source words.

Modern systems (Google Translate, DeepL) use transformer models.

Speech Recognition and Synthesis

Speech Recognition: Convert audio to text.

- Acoustic model: features from audio $\rightarrow$ phonemes.
- Language model: sequence of words, penalizes unlikely sentences.
- Decoding: find best word sequence given audio.

Speech Synthesis (TTS): Convert text to audio.

- Text analysis: parse input, normalize abbreviations.
- Linguistic feature extraction: phoneme, duration, pitch.
- Vocoder: generate waveform from features (can sound robotic or natural).

Modern methods: Neural networks and end-to-end learning improve naturalness and robustness.

NLP Challenges and Limitations

Fundamental issues:

- **Ambiguity:** Words and sentences have multiple meanings.
- **Context dependence:** Meaning relies on surrounding text and world knowledge.
- **Idioms & Figurative language:** "raining cats and dogs" is not literal.
- **Multilingual:** Each language has unique structure and resources.

Data challenges:

- Labeled data is expensive to create.
- Imbalanced classes (e.g., rare entities).
- Domain shift: model trained on news fails on social media.

Large Language Models (LLMs) have dramatically improved performance through scale and pretraining.

The Role of Language Models

Language models: Predict next word given context.

$$P(\text{word}_t | \text{word}_{1:t-1})$$

Impact:

- Foundation for many NLP tasks (translation, question answering).
- Transfer learning: pretrain on large text corpora, fine-tune on specific tasks.
- In-context learning: LLMs adapt to new tasks with few examples.

Examples:

- GPT models (autoregressive): Generate coherent long-form text.
- BERT (bidirectional): Better at understanding, classification, NER.
- Transformer architecture: Parallel processing, self-attention over context.

Tradeoff: Larger models are more capable but require more compute and

Summary and Next Steps

What we covered:

- Preprocessing: tokenization, normalization, stemming.
- BoW representation: efficient, simple, limited.
- Sentiment analysis: lexicon-based and learning-based approaches.
- Applications: chatbots, translation, speech, and more.

Key takeaway: NLP is a spectrum from simple pattern-matching to complex neural models.

Up next (Lecture 6): Introduction to Neural Networks — the engine behind modern NLP.

- Biological inspiration and the perceptron.
- Multilayer networks and backpropagation.